Titulo del Trabajo: Aprendizaje por Refuerzo

Autor1
jesus.guiradop@um.es

Autor2
antonioluis.sanchezt@um.es

Autor3
victoremilio.vicenteg@um.es

15 de marzo de 2026

Resumen

Los experimentos con el bandido de k-brazos se ejecutaron sobre un horizonte de 1000 pasos y 500 iteraciones independientes para asegurar la validez estadística, utilizando 10 brazos con distribuciones normal, binomial y de Bernoulli. Se evaluaron configuraciones específicas como $\epsilon \in \{0,0{,}01,0{,}1\}$ para el método ϵ-greedy, temperaturas de $\tau \in \{0{,}2,0{,}4,1{,}0\}$ en Softmax y coeficientes de exploración de $c \in \{0{,}0,1{,}41,4{,}0\}$ en UCB1. Los resultados destacaron a UCB1 con un coeficiente de 1.41 como la opción más robusta en el bandido Normal, UCB1 con coeficiente 4.0 para el bandido Binomial y ϵ-greedy con épsilon 0.1 para el algoritmo Bernoulli, mostrando una mayor tendencia a converger hacia el brazo óptimo en estos escenarios. Por otra parte, Softmax mostró dificultades para discriminar entre las mejores opciones.

En el caso del Blackjack, se diseñó una búsqueda de parámetros (grid search) evaluando valores de α entre 0.01 y 0.05, factores de descuento γ de hasta 1.0 y niveles de exploración ϵ entre 0.01 y 0.2. Tras un entrenamiento intensivo de 100.000 episodios, se determinó que los métodos de Diferencias Temporales, especialmente Q-learning, son superiores a los de Monte Carlo en este entorno. Se identificó que una exploración mínima ($\epsilon = 0{,}01$) y una tasa de aprendizaje fija de 0.05 son las configuraciones óptimas para manejar la alta varianza del juego y la brevedad de sus episodios.

Finalmente, las pruebas en Flappy Bird compararon la aproximación lineal de SARSA mediante Tile Coding frente a modelos de aprendizaje profundo con DQN. Se analizó el impacto de la arquitectura de la red, concluyendo que una configuración de tres capas con 128 nodos es la más eficiente, y se exploraron técnicas como el uso de n-pasos y la agregación de estados (frame stacking). Sorprendentemente, los experimentos indicaron que aumentar la complejidad temporal con múltiples pasos (n >1) o memoria de estados (k >1) introducía ruido innecesario, en este entorno las arquitecturas de DQN más simples son las que logran las mayores puntuaciones y estabilidad.

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.2. Desarrollo . . . 3

1.3. Algoritmos . . . 4

1.3.1. Epsilon-greedy . . . 4

1.3.2. Softmax . . . 4

1.3.3. UCB1 . . . 4

1.4. Evaluación/Experimentos . . . 5

1.4.1. Bandido Normal . . . 5

1.4.2. Bandido Binomial . . . 7

1.4.3. Bandido Bernoulli . . . 8

1.5. Conclusiones . . . 9

1.5.1. Resumen de los resultados . . . 10

1.5.2. Limitaciones del estudio y posibles mejoras futuras . . . 10

1.5.3. Reflexión sobre la importancia del trabajo y su impacto en el campo del aprendizaje por refuerzo . . . 10

1.5.4. Líneas futuras de estudio . . . 10

2. Aprendizaje en Entornos Complejos: Técnicas Tabulares **11**

2.1. Introducción . . . 11

2.2. Desarrollo . . . 11

2.2.1. Descripción del problema . . . 11

2.2.2. Representación formal del problema . . . 12

2.2.3. Contexto y abordaje del problema . . . 12

2.2.4. Trabajos relacionados y aportes originales . . . 13

2.3. Algoritmos . . . 13

2.3.1. Métodos de Monte Carlo . . . 13

2.3.2. Métodos de Diferencias Temporales . . . 14

2.4. Evaluación/Experimentos . . . 14
2.4.1. Rendimiento baseline: Agente aleatorio . . . 14
2.4.2. Influencia de parámetros . . . 15
2.4.3. Experimentos con ϵ-decaimiento . . . 15
2.4.4. Análisis de la cobertura de estados . . . 16
2.4.5. Comparativa global . . . 17
2.5. Conclusiones . . . 17
2.5.1. Conclusiones y resumen del trabajo . . . 17
2.5.2. Limitaciones y vías futuras . . . 18

3. Aprendizaje en Entornos Complejos **19**
3.1. Introducción . . . 19
3.2. Desarrollo . . . 19
3.2.1. Descripción del problema . . . 19
3.2.2. Representación formal del problema . . . 19
3.2.3. Trabajos Relacionados . . . 20
3.2.4. Aportes del trabajo . . . 20
3.3. Algoritmos . . . 20
3.3.1. Baseline aleatorio . . . 20
3.3.2. SARSA semi-gradiente (on-policy) con Tile Coding . . . 20
3.3.3. Deep Q-Learning . . . 21
3.3.4. Agregación de estados . . . 21
3.4. Evaluación/Experimentos . . . 21
3.4.1. Rendimiento baseline: Agente aleatorio . . . 21
3.4.2. Influencia de Tile Coding en SARSA . . . 22
3.4.3. Influencia de parámetros Deep Q-Learning . . . 22
3.4.4. Influencia de la arquitectura de la red en Deep Q-Learning . . . 22
3.4.5. Agregación de estados . . . 23
3.4.6. n-Pasos . . . 23
3.5. Conclusiones y Vías futuras . . . 23
3.5.1. Conclusiones . . . 23
3.5.2. Limitaciones y vías futuras . . . 24

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

Este problema consiste en elegir repetidamente una acción entre varias (brazos) en las cuales cada acción tendrá una recompensa desconocida. El objetivo de este problema es maximizar las recompensas, por lo que surge el clásico problema de **exploración** (probar distintas acciones) vs **explotación** (elegir únicamente la acción que más recompensas da).

Esto hace que sea un problema muy relevante en la forma de comprender cómo los agentes inteligentes toman decisiones bajo incertidumbre. Además, sirve como base teórica para métodos más avanzados de aprendizaje por refuerzo utilizados en aplicaciones reales.

Con este trabajo se pretende hacer un estudio comparativo de los distintos algoritmos con sus respectivas configuraciones, desde las más explotadoras a las más exploradoras, analizando sus conductas en distintas gráficas, que iremos analizando en la sección de Evaluación/Experimentos.

La estructura de trabajo será la siguiente:

- Introducción: descripción del problema, su importancia, el objetivo y la organización del documento.
- Desarrollo: Definicion formal, enfoques, contexto y antecedentes, explicacion de metodos y justificar la diferencia de este trabajo con enfoques previos
- Algoritmos: Explicar los algoritmos, pseudocodigo y justificar su elección y aplicabilidad.
- Evaluación/Experimentos: Entornos ejecutados, métricas utilizadas y análisis crítico de los resultados presentados en gráficas.
- Conclusiones: Resumen de resultados, limitaciones de estudio y posibles mejoras a futuro.

1.2. Desarrollo

El problema del bandido de k-brazos modela una situación de toma de decisiones secuencial bajo incertidumbre en un entorno estacionario. Se considera una máquina de k brazos (acciones), donde cada brazo $i \in 1, ..., k$ está asociado a una recompensa desconocida P_i.

En cada instante temporal t, el agente:

- Selecciona una acción $a_t \in 1, ..., k$.
- Recibe una recompensa $r_t \sim P_{at}$.

El objetivo es maximizar la recompensa acumulada a lo largo de un horizonte temporal T:

$$\sum_{T=1}^{T} r_t$$

Dado que las distribuciones de recompensa son desconocidas, el agente debe equilibrar la exploración de brazos poco conocidos con la explotación de aquellos que han mostrado buenos resultados.

En la literatura se han propuesto diversas estrategias para abordar el problema del bandido de k-brazos, cuyo objetivo es equilibrar exploración y explotación al seleccionar acciones con recompensas desconocidas. Entre los enfoques más conocidos se encuentran los métodos acción-valor, que estiman el valor esperado de cada acción a partir de las recompensas observadas, así como estrategias como ϵ-greedy, Softmax o UCB, que incorporan distintos mecanismos para gestionar el compromiso entre explorar nuevas acciones y explotar las que han mostrado mejores resultados. Más información sobre los algoritmos en la sección de Algoritmos.

Como contexto, el problema del bandido de k-brazos es un modelo clásico de aprendizaje por refuerzo, en el cual un agente toma decisiones secuenciales bajo incertidumbre. Conceptos fundamentales son: **acción**, **recompensa**, **exploración** y **explotación**. Además, el estudio considera distintos tipos de bandido, **normal**, **binomial** y **Bernoulli**, lo que permite evaluar el desempeño de los algoritmos bajo diferentes distribuciones de recompensa. Este tipo de problemas tiene aplicaciones en situaciones de la vida real donde se deben tomar decisiones sin conocer previamente los resultados, como en sistemas de recomendación, estrategias de marketing o juegos.

Para esto hemos utilizado los algoritmos ϵ-greedy, softmax y UCB1, donde en cada uno de ellos analizamos 3 configuraciones, desde más explotadoras a más exploradoras. Para ello analizaremos estos algoritmos sobre gráficas donde tenemos recompensas promedio vs pasos de tiempo, selección de brazo óptimo (%) vs pasos de tiempo, regret acumulado vs pasos de tiempo y selección de cada brazo.

por lo tanto en este trabajo se tratará sobre evaluar distintas configuraciones de cada algoritmo analizándolos sobre múltiples métricas, que irán cambiando según el tipo de bandido que estemos estudiando: normal, binomial y Bernoulli.

1.3. Algoritmos

1.3.1. Epsilon-greedy

Algorithm 1 Algoritmo ϵ-Greedy

Require: Número de acciones T, número de opciones (brazos) K, parámetro de exploración ϵ
Ensure: Recompensas acumuladas R_k y contadores N_k para cada opción k

```
Inicializar recompensas acumuladas R_k ← 0 y contador de elecciones N_k ← 0 para cada opción k = 1, 2, ..., K
Seleccionar cada opción k una vez para inicializar la recompensa
for t = K + 1 hasta T do
    Generar un número aleatorio p entre 0 y 1
    if p < ε then
        Exploración: Seleccionar una opción k aleatoria entre 1 y K
    else
        Explotación: Seleccionar la opción k que maximiza R_k / N_k (opción con mayor recompensa promedio)
    end if
    Realizar la acción k y observar la recompensa r_k
    Actualizar la recompensa acumulada: R_k ← R_k + r_k
    Actualizar el contador de elecciones: N_k ← N_k + 1
end for
```

Como se puede ver en el pseudocódigo **Algortihm 1**, al inicio se selecciona cada brazo una vez para tener una recompensa inicial, luego entra en un bucle $t = k + 1$ hasta T, en el que se genera un número aleatorio p entre 0 y 1, si este valor es menor que ϵ, seleccionará un brazo de forma aleatorio entre 1 y K. En caso contrario, el algoritmo selecciona el brazo con mayor recompensa promedio estimada, se realiza la acción y se observa la recompensa y se actualiza la recompensa acumulada y el contador del brazo elegido.

Se utiliza el algoritmo ϵ-greedy porque está basado en el algoritmo greedy, que selecciona siempre el brazo con mayor recompensa promedio estimada. Resulta interesante analizar cómo cambia su comportamiento al introducir una probabilidad de exploración ϵ. Para este estudio se emplean los valores $\epsilon \in \{0, 0{,}01, 0{,}1\}$.

1.3.2. Softmax

Nos centraremos en el pseudocodigo de **Algortihm 2**. Para cada acción se establece una recompensa promedio inicial y se establece un valor $\tau > 0$. En cada instante de tiempo, se calculan las probabilidades de selección para cada acción, en el que cuanto mayor sea el valor de τ más uniforme será la distribución de probabilidades, promoviendo que el algoritmo adquiera una actitud exploradora; por el contrario, cuanto menor sea el valor de τ se distribuirán mayor probabilidades a las acciones con mayor recompensa promedio, favoreciendo una actitud explotadora. Una vez calculadas las probabilidades, se selecciona una acción de acuerdo con la distribución, se recibe la recompensa correspondiente y se actualiza la recompensa promedio del brazo seleccionado. Este proceso se repite en cada instante de tiempo hasta completar todos los pasos.

Algorithm 2 Algoritmo Softmax

Require: Número de acciones T, número de opciones (brazos) K, parámetro de temperatura $\tau > 0$
Ensure: Recompensas acumuladas R_k y recompensas promedio $Q(a)$ para cada brazo a

```
Inicializar para cada acción a: Q(a) ← 0
for t = 1 hasta T do
    for k = 1 hasta K do
        Calcular las probabilidades de selección:
        π_t(a) = e^{Q(a)/τ} / Σ_{b=1}^{K} e^{Q(b)/τ}
    end for
    Seleccionar una acción A_t de acuerdo con la distribución de probabilidad π_t(a)
    Recibir la recompensa R_t correspondiente a la acción A_t
    Actualizar la recompensa promedio del brazo seleccionado:
        Q(a) ← Q(a) + (1/N(a)) (R_t − Q(a))
    Actualizar el contador de selecciones N(a) ← N(a)+1
end for
```

Se utiliza el algoritmo Softmax porque asigna probabilidades de selección a los brazos en función de sus recompensas promedio estimadas. Esto permite analizar su comportamiento a distintos niveles de temperatura, que controlan el equilibrio entre exploración y explotación. Para este estudio se emplean los valores $\tau \in \{0{,}2, 0{,}5, 1{,}0\}$.

1.3.3. UCB1

En el pseudocódigo **Algortihm 3** se inicializan las recompensas promedio y el número de selecciones de cada brazo a 0. A continuación, se realiza una selección inicial de cada brazo (exploración inicial), lo que permite obtener una primera estimación de la recompensa promedio y establecer el contador de selecciones de cada brazo a 1. Posteriormente, para cada instante de tiempo, se selecciona la acción que maximiza la suma entre la recompensa promedio estimada del brazo y un término de exploración. Este término está formado por el coeficiente c multiplicado por la raíz cuadrada del logaritmo neperiano de t dividido entre el número de veces que se ha seleccionado ese brazo. Una vez seleccionada la acción, se ejecuta y se observa la recompensa obtenida. A continuación, se actualiza el contador de selecciones y la recompensa promedio estimada del brazo seleccionado. Este proceso se repite iterativamente hasta completar todos los instantes de tiempo.

El coeficiente c controla el grado de exploración del algoritmo: valores mayores favorecen explorar brazos menos probados, mientras que valores menores priorizan la explotación de los brazos con mayor recompensa promedio. A medida que el algoritmo se ejecuta, el término de exploración disminuye para los brazos que ya han sido seleccionados muchas veces, lo que provoca que el algoritmo explore progresivamente menos y explote más las acciones con mejores recompensas. De esta forma, UCB1 se autorregula en su exploración, a diferencia de algoritmos como Softmax o ϵ-greedy, en los que la exploración

Algorithm 3 Algoritmo UCB1 para el Problema del Bandido Multibrazo

Require: Número de acciones T, número de opciones (brazos) K, coeficiente de exploración c

Ensure: Recompensas acumuladas R_k y contadores N_k para cada brazo k

1: Inicializar $Q(a) \leftarrow 0$ y $N(a) \leftarrow 0$ para cada brazo $a = 1, ..., K$
2: Seleccionar cada brazo una vez y actualizar $Q(a)$ y $N(a)$ (**Necesario**)
3: **for** $t = K + 1$ hasta T **do**
4: Seleccionar la acción

$$A_t = \arg\max_a \left(Q(a) + c\sqrt{\frac{\ln t}{N(a)}} \right)$$

5: Ejecutar la acción A_t y observar la recompensa R_t
6: Actualizar el contador: $N(A_t) \leftarrow N(A_t) + 1$
7: Actualizar la recompensa promedio:

$$Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$$

8: **end for**

depende de un parámetro fijo. Para este estudio se emplean los valores $c \in \{0{,}0, 1{,}41, 4{,}0\}$.

1.4. Evaluación/Experimentos

El código ha sido desarrollado en python utilizando Visual Studio Code y Google Colab.

Las librerías y módulos utilizados son: `NumPy`, `List` de `typing`; y los módulos implementados para este trabajo `Algorithm`, `EpsilonGreedy`, `Softmax` y `UCB1` de la carpeta `algorithms`; `ArmNormal`, `ArmBinomial`, `ArmBernoulli` y `Bandit` de la carpeta `arms`; y las funciones `plot_average_rewards`, `plot_optimal_selections`, `plot_regret` y `plot_arm_statistics` de la carpeta `plots`.

En cuanto a la configuración de los experimentos, se ejecutarán sobre 1000 pasos en 500 runs en el que calculamos el promedio. Estos experimentos serán ejecutados sobre 10 brazos según el tipo de bandido, ya sea Normal, Binomial o Bernoulli:

- **Normal**: Sigue una distribución normal donde el parámetro μ es la media y σ es la desviación estándar, siendo este último mayor que 0 debido a que negativo no tiene sentido por definición, y tampoco puede ser 0 debido a la función de densidad de la distribución normal que divide entre σ, lo que en la práctica no tendría sentido ya que devolvería siempre el mismo valor. En el experimento, la media de cada brazo se inicializa aleatoriamente con valores entre 1 y 10, mientras que la desviación estándar se fija en $\sigma = 1$. Por lo tanto, la recompensa obtenida al seleccionar el brazo a en el instante t se genera como $R_t(a) \sim N(\mu_a, 1)$. Donde el segundo parámetro indica la varianza σ^2.
- **Binomial**: En cuanto al bandido de tipo binomial, viene dado por el número de acciones realizadas n, que darán lugar al número total de éxitos con una probabilidad p, por lo cual el valor esperado vendrá dado de $n * p$. Para este experimento, el número de acciones de cada brazo, n, hemos establecido 100 y las probabilidades p serán aleatorias desde 0.1 a 0.9.
- **Bernoulli**: Este tipo de bandidos es un caso particular del tipo binomial, la única diferencia entre estos dos es que bernoulli tiene el parámetro del número de acciones fijo, con $n = 1$, además de que las probabilidades p suelen ser más pequeñas. El valor esperado vendrá dado por p y tendrá unos valores entre 0.05 y 0.5.

Las métricas utilizadas para los experimentos son:

- **plot_average_rewards**: Muestra la recompensa media obtenida por cada algoritmo en cada instante de tiempo. Para calcularla, se registra la recompensa obtenida en cada paso y se promedia sobre todas las ejecuciones del experimento. Esto permite observar cómo evoluciona el rendimiento de los algoritmos a lo largo del tiempo.
- **plot_optimal_selections**: En cada paso se revisa el brazo seleccionado, si lo es se suma 1 al contador y luego se promedia entre todas las ejecuciones del experimento.
- **plot_regret**: Para calcular el regret acumulado se utiliza la diferencia en cada instante de tiempo entre el valor esperado del brazo óptimo q^* y la recompensa obtenida r_t en el brazo seleccionado. El regret acumulado hasta el paso T se define como:

$$R_T = q^* T - \sum_{t=1}^{T} r_t$$

  Al promediar los resultados sobre múltiples ejecuciones del experimento, esta expresión aproxima el arrepentimiento esperado presentado en las transparencias del profesor
- **plot_arm_statistics**: En esta métrica se muestra, para cada algoritmo, cuántas veces se seleccionó cada brazo y la recompensa acumulada de cada uno. Permite ver qué brazos prefiere cada algoritmo y cómo se distribuyen las recompensas. Además de que se señalará en el histograma con color verde el brazo óptimo.

Ahora procederemos a analizar los resultados, con las métricas explicadas dependiendo del tipo de bandido.

1.4.1. Bandido Normal

Recompensa promedio

Como se puede observar en la figura 1.1, el algoritmo ϵ-greedy obtiene mejores resultados cuando utiliza poca o ninguna exploración. Esto indica que el algoritmo identifica rápidamente los brazos con mayor recompensa y los explota con mayor frecuencia. En cambio, con $\epsilon = 0{,}1$ la recompensa promedio es menor, ya que el algoritmo selecciona brazos aleatorios en un 10 % de los casos, lo que introduce pérdidas acumuladas al elegir brazos subóptimos.

En el caso de Softmax, temperaturas más altas (τ) producen una mayor recompensa promedio. Esto ocurre porque la distribución de probabilidades sobre los brazos se vuelve más

images/normal_recompensapromedio_epsilon.png

images/normal_recompensapromedio_softmax.png

images/normal_recompensapromedio_ucb1.png

Figura 1.1: Recompensas promedio bandido normal

uniforme, favoreciendo la exploración y permitiendo identificar antes el brazo óptimo.

En cuanto a UCB1, valores altos del coeficiente como $c = 4$ provocan una convergencia más lenta debido a la mayor exploración inicial. En cambio, valores menores como $c = 0$ y $c = 1{,}41$ alcanzan antes recompensas altas, aunque con el tiempo todas las configuraciones tienden a converger hacia valores similares.

Selección de brazo óptimo

Como se observa en la figura 1.2, en el algoritmo ϵ-greedy la configuración $\epsilon = 0$ no mejora el porcentaje de selección del brazo óptimo, ya que al no existir exploración el algoritmo no vuelve a evaluar otros brazos tras la estimación inicial. En cambio, $\epsilon = 0{,}01$ y $\epsilon = 0{,}1$ muestran una mejora progresiva, puesto que la exploración permite actualizar las estimaciones de recompensa y aumentar la probabilidad de seleccionar el brazo óptimo.

En el algoritmo Softmax, la configuración $\tau = 1{,}0$ presenta el mayor crecimiento en la selección del brazo óptimo, ya que una mayor temperatura favorece la exploración al generar una distribución de probabilidades más uniforme entre los brazos. No obstante, el porcentaje alcanzado sigue siendo inferior al obtenido por ϵ-greedy y UCB1.

En UCB1 se observa que con $c = 1{,}41$ el algoritmo alcanza rápidamente porcentajes cercanos al 100 % de selección del brazo óptimo, mostrando un buen equilibrio entre exploración y explotación. Con $c = 0$ el algoritmo apenas explora y se estanca, mientras que con $c = 4{,}0$ la mayor exploración inicial provoca una convergencia más lenta, aunque finalmente también alcanza valores elevados.

images/normal_seleccionbrazooptimo_epsilon.png

images/normal_seleccionbrazooptimo_softmax.png

images/normal_seleccionbrazooptimo_ucb1.png

Figura 1.2: Selección de brazo optimo del bandido normal

Regret acumulado

Como se observa en la figura 1.3, el algoritmo ϵ-greedy obtiene los mejores resultados con $\epsilon = 0$ y $\epsilon = 0{,}01$, ya que identifica rápidamente brazos con altas recompensas sin realizar exploraciones innecesarias. Además, $\epsilon = 0{,}01$ muestra un regret acumulado ligeramente menor, probablemente debido al aumento progresivo en la selección del brazo óptimo.

En el caso de Softmax, el regret acumulado es considerablemente mayor, lo que indica que el algoritmo selecciona con frecuencia brazos alejados del óptimo. No obstante, la temperatura $\tau = 1{,}0$ presenta el mejor comportamiento dentro de este algoritmo, mostrando una ligera mejora asociada al aumento de la recompensa promedio y del porcentaje de selección del brazo óptimo.

En UCB1, la configuración que mejor funciona es $c = 1{,}41$, ya que alcanza rápidamente altos valores de selección del brazo óptimo.

Test estadístico de cada brazo

En la figura 1.4 se muestran las estadísticas por brazo de los algoritmos que mejor seleccionan el brazo óptimo: ϵ-Greedy con $\epsilon = 0{,}01$, Softmax con $\tau = 1{,}0$ y UCB1 con $c = 1{,}41$. Los histogramas indican cuántas veces se seleccionó cada brazo y la recompensa acumulada obtenida. El brazo óptimo, que en este caso es el brazo 7, se resalta en verde.

images/normal_regret_epsilon.png

images/normal_regret_softmax.png

images/normal_regret_ucb1.png

Figura 1.3: Regret acumulado del bandido normal

images/normal_brazo_epsilon.png images/normal_brazo_softmax.png images/normal_brazo_ucb1.png

Figura 1.4: Test estadísticos del bandido normal

1.4.2. Bandido Binomial

Recompensa promedio

En la figura 1.5, el algoritmo ϵ-greedy muestra un comportamiento similar al observado en el bandido normal. Las configuraciones $\epsilon = 0$ y $\epsilon = 0{,}01$ alcanzan las mayores recompensas promedio, lo que indica que el algoritmo identifica con facilidad el brazo con mayor probabilidad de éxito. En cambio, con $\epsilon = 0{,}1$ la recompensa promedio es menor, ya que el mayor nivel de exploración provoca la selección más frecuente de brazos con menor recompensa esperada.

En el caso de Softmax, las temperaturas más bajas ($\tau = 0{,}2$ y $\tau = 0{,}5$) obtienen una recompensa promedio ligeramente superior a $\tau = 1{,}0$, lo que sugiere que una exploración demasiado alta dificulta la explotación de los brazos con mayor recompensa.

En cuanto a UCB1, los tres valores del coeficiente c alcanzan prácticamente la misma recompensa promedio, indicando que el algoritmo identifica con facilidad el brazo óptimo y termina explotándolo independientemente del nivel de exploración.

images/binomial_recompensa_epsilon.png

images/binomial_recompensa_softmax.png

images/binomial_recompensa_ucb1.png

Figura 1.5: Recompensas promedio bandido binomial

Selección de brazo óptimo

En la figura 1.6, el algoritmo ϵ-greedy muestra que $\epsilon = 0$ y $\epsilon = 0{,}01$ alcanzan valores cercanos al 100 % de selección del brazo óptimo, indicando que el algoritmo identifica rápidamente el brazo con mayor probabilidad de éxito y lo explota durante la mayor parte del tiempo. En cambio, con $\epsilon = 0{,}1$ el porcentaje es menor debido al mayor nivel de exploración.

En el algoritmo Softmax, el porcentaje de selección del brazo óptimo se mantiene bajo y apenas crece con el tiempo. Esto indica que el algoritmo no logra concentrar suficiente probabilidad en el brazo óptimo, incluso con temperaturas más altas ($\tau = 1{,}0$), donde la distribución entre los brazos es más uniforme.

En UCB1 se observa que el algoritmo identifica el brazo óptimo con cualquier valor del coeficiente c. Tras una fase inicial de exploración, converge rápidamente hacia dicho brazo y lo selecciona la mayor parte del tiempo.

Regret acumulado

En la figura 1.7, el algoritmo ϵ-greedy muestra que las configuraciones $\epsilon = 0$ y $\epsilon = 0{,}01$ acumulan menos regret, ya que presentan un alto porcentaje de selección del brazo óptimo cercano al 95 %. No obstante, al no alcanzar el 100 %, el regret sigue creciendo con el tiempo, siendo $\epsilon = 0$ el que lo hace más lentamente debido a su carácter más explotador.

En el caso de Softmax, las tres temperaturas presentan valores de regret acumulado muy elevados, lo que evidencia un bajo rendimiento del algoritmo en este experimento.

images/binomial_optimo_epsilon.png

images/binomial_optimo_softmax.png

images/binomial_optimo_ucb1.png

Figura 1.6: Selección de brazo optimo del bandido binomial

images/binomial_regret_epsilon.png

images/binomial_regret_softmax.png

images/binomial_regret_ucb1.png

Figura 1.7: Regret acumulado del bandido binomial

Por su parte, UCB1 es el algoritmo que menos regret acumula, especialmente con $c = 4{,}0$. Esto sugiere que una mayor exploración permite identificar antes el brazo óptimo y reducir el regret acumulado.

images/binomial_brazo_epsilon.png images/binomial_brazo_softmax.png

Figura 1.8: Test estadísticos del bandido binomial

Test estadístico de cada brazo

En la figura 1.8 se muestran las estadísticas por brazo de los algoritmos que mejor seleccionan el brazo óptimo: ϵ-Greedy con $\epsilon = 0$, Softmax con $\tau = 0{,}5$ y UCB1 con $c = 4{,}0$. Los histogramas indican cuántas veces se seleccionó cada brazo y la recompensa acumulada obtenida. El brazo óptimo, que en este caso es el brazo 1, se resalta en verde.

1.4.3. Bandido Bernoulli

Recompensa promedio

En la figura 1.9, el algoritmo ϵ-greedy muestra recompensas promedio muy similares para los tres valores de ϵ. Esto sugiere que, en este entorno Bernoulli, el algoritmo puede encontrar brazos con buenas probabilidades de éxito incluso con distintos niveles de exploración. De hecho, $\epsilon = 0{,}1$ presenta en algunos tramos valores ligeramente superiores, lo que indica que una mayor exploración puede ayudar a descubrir mejores brazos.

En el algoritmo Softmax, la temperatura más baja ($\tau = 0{,}2$) obtiene la mayor recompensa promedio, lo que indica que en este entorno resulta beneficioso concentrar mayor probabilidad en los brazos con mejores estimaciones. Temperaturas más altas generan una distribución más uniforme y favorecen la selección de brazos con menor recompensa.

En UCB1 se observa que la configuración $c = 1{,}41$ presenta un crecimiento progresivo de la recompensa promedio. Aunque $c = 0$ obtiene mejores resultados al inicio, su carácter explotador provoca un estancamiento, mientras que el equilibrio entre exploración y explotación de $c = 1{,}41$ permite encontrar recompensas mayores con el tiempo.

Selección de brazo óptimo

En la figura 1.10, el algoritmo ϵ-greedy muestra que el porcentaje de selección del brazo óptimo aumenta con el nivel de exploración. En particular, $\epsilon = 0{,}1$ alcanza el mayor valor, lo que sugiere que la exploración adicional permite corregir estimaciones iniciales bajas del brazo óptimo y seleccionarlo con mayor frecuencia.

En el algoritmo Softmax se observa un ligero crecimiento inicial, tras el cual las tres configuraciones se estabilizan en valores bajos. La temperatura $\tau = 0{,}2$ obtiene el mejor resultado, indicando que una estrategia más explotadora favorece la selección del brazo con mayor recompensa estimada, mientras que temperaturas más altas distribuyen la probabilidad entre más brazos.

images/bernoulli_recompensa_epsilon.png

images/bernoulli_recompensa_softmax.png

images/bernoulli_recompensa_ucb1.png

Figura 1.9: Recompensa promedio del bandido Bernoulli

images/bernoulli_optimo_epsilon.png

images/bernoulli_optimo_softmax.png

images/bernoulli_optimo_ucb1.png

Figura 1.10: Selección de brazo óptimo del bandido Bernoulli

En UCB1, la configuración $c = 0{,}0$ converge rápidamente hacia un brazo concreto pero se estanca, mientras que $c = 1{,}41$ muestra un crecimiento progresivo. Esto indica que un equilibrio entre exploración y explotación permite aumentar el porcentaje de selección del brazo óptimo.

Regret acumulado

En la figura 1.11, en ϵ-greedy, el crecimiento del regret acumulado disminuye al aumentar ϵ, siendo $\epsilon = 0{,}1$ la configuración con menor regret, ya que la exploración permite identificar el brazo óptimo y evitar quedarse en brazos subóptimos.

En Softmax, la temperatura más baja ($\tau = 0{,}2$) acumula menos regret al favorecer la explotación, mientras que temperaturas más altas distribuyen la selección entre más brazos, incrementando el regret.

En UCB1, $c = 0{,}0$ presenta el menor regret acumulado, indicando que una estrategia más explotadora resulta favorable en este entorno, mientras que mayores valores de c aumentan la exploración y, con ello, el regret.

Test estadístico de cada brazo

En la figura 1.12 se muestran las estadísticas por brazo de los algoritmos que mejor seleccionan el brazo óptimo: ϵ-Greedy con $\epsilon = 0{,}1$, Softmax con $\tau = 0{,}2$ y UCB1 con $c = 0{,}0$. Los histogramas indican cuántas veces se seleccionó cada brazo y la recompensa acumulada obtenida. El brazo óptimo, que en este caso es el brazo 1, se resalta en verde.

images/bernoulli_regret_epsilon.png

images/bernoulli_regret_softmax.png

images/bernoulli_regret_ucb1.png

Figura 1.11: Regret acumulado del bandido Bernoulli

1.5. Conclusiones

Para las familias ϵ-greedy, la gráfica más relevante es la gráfica de % de selección de brazo óptimo. Esto se debe a que refleja

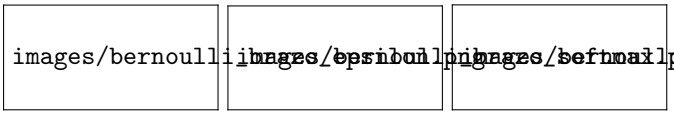

Figura 1.12: Test estadísticos del bandido Bernoulli

de manera directa la capacidad del algoritmo para identificar y explotar el brazo con mayor recompensa esperada. En comparación, la recompensa promedio suele ser más uniforme y menos sensible a los cambios en ϵ.

1.5.1. Resumen de los resultados

Bandido Normal

En el bandido Normal se observa que ϵ-greedy funciona mejor con un comportamiento principalmente explotador. Con $\epsilon = 0$ o $\epsilon = 0{,}01$ el algoritmo identifica rápidamente los brazos con mayor recompensa promedio y converge hacia el brazo óptimo.

El algoritmo Softmax mejora su rendimiento con temperaturas más altas, ya que una mayor exploración le permite descubrir brazos con mejor recompensa promedio.

En el caso de UCB1, los coeficientes bajos o moderados ofrecen los mejores resultados, permitiendo identificar el brazo óptimo sin realizar exploraciones excesivas.

Comparando los tres algoritmos, UCB1 con $c = 1{,}41$ es el que muestra el mejor rendimiento global, con alta recompensa promedio, bajo regret acumulado y un elevado porcentaje de selección del brazo óptimo.

Bandido Binomial

En el bandido Binomial, ϵ-greedy también muestra un mejor rendimiento con configuraciones más explotadoras, ya que tras las primeras estimaciones el algoritmo identifica con facilidad los brazos con mayor probabilidad de éxito.

Softmax presenta un rendimiento inferior en este entorno, con menor recompensa promedio, bajo porcentaje de selección del brazo óptimo y mayor regret acumulado.

Por su parte, UCB1 obtiene muy buenos resultados en todas sus configuraciones, destacando especialmente la configuración $c = 4{,}0$, que presenta el menor regret acumulado.

En conjunto, UCB1 con $c = 4{,}0$ es el algoritmo que mejor funciona en este experimento, ya que ofrece la mayor recompensa promedio, el menor regret y el mayor porcentaje de selección del brazo óptimo.

Bandido Bernoulli

En el bandido Bernoulli, ϵ-greedy obtiene mejores resultados con mayor exploración, ya que esto facilita identificar el brazo óptimo y reduce el regret acumulado.

En Softmax, las temperaturas bajas producen mejores resultados, ya que favorecen la explotación del brazo con mayor recompensa estimada.

En UCB1, los valores bajos de c muestran un mejor rendimiento, ya que reducen la exploración y favorecen la selección frecuente del brazo óptimo.

Comparando los algoritmos, Softmax es el que peor funciona en este entorno. En cambio, ϵ-greedy y UCB1 muestran resultados similares, aunque ϵ-greedy con $\epsilon = 0{,}1$ presenta una ligera ventaja al alcanzar un mayor crecimiento en la selección del brazo óptimo y un menor regret acumulado.

1.5.2. Limitaciones del estudio y posibles mejoras futuras

Este estudio presenta algunas limitaciones. En primer lugar, los experimentos se han realizado con un número concreto de configuraciones de los algoritmos y con un número limitado de pasos de tiempo, por lo que diferentes parámetros o ejecuciones más largas podrían producir resultados ligeramente distintos. Además, el análisis se ha centrado únicamente en tres algoritmos clásicos de resolución del problema del bandido de k-brazos.

Como posibles mejoras futuras, sería interesante ampliar el estudio incluyendo otras estrategias como ϵ-decaimiento o variantes más avanzadas como UCB2. También podría ampliarse el número de experimentos o analizar otros tipos de distribuciones de recompensa para evaluar cómo se comportan los algoritmos en entornos más variados.

1.5.3. Reflexión sobre la importancia del trabajo y su impacto en el campo del aprendizaje por refuerzo

El problema del bandido de k-brazos es fundamental en aprendizaje por refuerzo, ya que permite estudiar el equilibrio entre exploración y explotación. Los experimentos realizados muestran cómo algoritmos como ϵ-greedy, Softmax y UCB1 presentan distintos comportamientos según la exploración y el tipo de distribución de recompensas, ayudando a comprender sus ventajas, limitaciones y su ajuste en problemas reales de aprendizaje por refuerzo.

1.5.4. Líneas futuras de estudio

Como línea futura de investigación, sería interesante estudiar la aplicabilidad de los algoritmos del bandido de k-brazos en entornos dinámicos o no estacionarios, donde las probabilidades de éxito de los brazos cambian con el tiempo. También podría explorarse la combinación de estos algoritmos con técnicas de aprendizaje profundo para abordar problemas con grandes espacios de acción o recompensas más complejas. Otra dirección posible es investigar cómo diferentes métodos de selección de brazo óptimo afectan la robustez y eficiencia de los algoritmos en escenarios con incertidumbre o ruido elevado, aportando así una visión más general del rendimiento de estas estrategias en contextos prácticos de aprendizaje por refuerzo.

Capítulo 2

Aprendizaje en Entornos Complejos: Técnicas Tabulares

2.1. Introducción

En Aprendizaje por Refuerzo, el enfoque del bandido de k-brazos nos permite explorar entornos de aprendizaje en los que el contexto es invariante durante el entrenamiento. Es decir, las distribuciones estadísticas de las recompensas de cada brazo no cambian en función del instante **t** en que nos encontremos. Sin embargo, esto no resulta un acercamiento realista a problemas reales. En la vida real, normalmente encontraremos problemas con contextos cambiantes en los que cada acción tomada por un agente en un instante **t** de tiempo provoque en el mismo un cambio de estado interno. Este nuevo estado, a su vez, determinará tanto el conjunto de acciones posibles a ejecutar como nuevas distribuciones posibles de recompensas según la nueva acción elegida en el instante **t+1**. Además, en este tipo de problemas tampoco conocemos el modelo que rige el entorno ni las posibles recompensas tras tomar cada acción y, por tanto, desconocemos la mejor decisión posible para cada estado. Por ello, debemos usar métodos basados en la experiencia del agente al interactuar con su entorno para poder modelar el proceso de decisión de Markov (MDP) subyacente al problema.

En este capítulo del documento, estudiaremos distintos métodos que nos permiten abordar problemas de aprendizaje mediante el uso de tablas para representar las funciones de evaluación de estados y acciones de un agente. Por tanto, con estas técnicas podremos tratar únicamente problemas con un número de estados y acciones discreto y finito, y normalmente no excesivamente grande (en términos computacionales).

Un claro ejemplo de este tipo de problemas es **el problema del Blackjack**. Se trata de un juego de apuestas en el que tendremos que intentar llegar a 21 con las cartas repartidas sin pasarnos al mismo tiempo que intentamos superar la mano del crupier. Este famoso juego de cartas resulta perfecto para intentar utilizar técnicas tabulares de aprendizaje por refuerzo para intentar modelar el proceso de decisión de Markov intrínseco al juego y encontrar el conjunto de decisiones óptimas para cada estado del agente. Los motivos principales son la sencillez del mismo y el acotado número de estados y acciones posibles dentro del problema. Además, como juego de azar que es, presenta también una serie de dificultades y complejidades específicas, que estudiaremos más adelante, que hacen que el aprendizaje del agente no sea demasiado sencillo e inmediato. Estas nos permitirán ver claramente las diferencias propias de cada tipo de algoritmo que usaremos para abordar este problema. No es casualidad, que sea uno de los problemas estudiados por Sutton y Barto en su libro introductorio al Aprendizaje por Refuerzo [8].

En este capítulo del documento haremos un estudio comparativo en el que analizaremos las diferencias propias entre los principales algoritmos de aprendizaje por refuerzo a la hora de abordar el problema del Blackjack. En concreto, haremos una comparación entre algoritmo basados en métodos de Monte Carlo (On-policy y Off-policy) y algoritmos basados en Diferencias Temporales (SARSA y Q-learning). Intentaremos reflejar claramente las fortalezas y debilidades de cada uno a la hora de enfrentarse a entornos como este haciendo un análisis exhaustivo del problema.

La estructura de trabajo será la siguiente:

- Desarrollo: Definición formal del problema del Blackjack, contexto, complejidades intrínsecas del problema y antecedentes y explicación de los métodos experimentales.
- Algoritmos: Explicación de los algoritmos utilizados en el estudio, analizando su pseudocódigo y como se aplican al problema.
- Evaluación/Experimentos: Presentación de los entornos ejecutados, las métricas utilizadas y análisis crítico de los resultados presentados en gráficas.
- Conclusiones: Resumen de resultados, limitaciones del estudio y posibles mejoras a futuro.

2.2. Desarrollo

2.2.1. Descripción del problema

El juego del Blackjack es un famoso juego de cartas basado en el azar que presenta el siguiente objetivo: **acercarse lo máximo posible sin pasarse a 21 con la mano de cartas repartida superando, al mismo tiempo, la mano de cartas del crupier**. Además, hay que tener en cuenta otras normas fundamentales del juego:

- Al comenzar el juego, el crupier reparte una mano de dos cartas al jugador. Al mismo tiempo, se reparte dos cartas a sí mismo de las cuales sólo mostrará una; la otra quedará boca abajo hasta el final de la ronda.
- En cada ronda, el jugador puede pedir más cartas, para intentar acercarse a 21 o plantarse con la suma total de

cartas actual. Si el jugador se pasa de 21, pierde instantáneamente, independientemente de las cartas que tenga el crupier. Al final de cada ronda, el crupier descubre la carta oculta y se sigue repartiendo cartas a sí mismo hasta alcanzar una suma igual o superior a 17.
- Todas las figuras (J,Q y K) tienen una puntuación de 10. Asimismo, los ases (A) pueden valer 1 ó 11, según le interese al jugador en cada momento. Se asume siempre la máxima suma para el jugador siempre que esté debajo de 21. El resto de cartas (2-10) tienen un valor igual a su número.
- Al final de cada ronda, mientras que no se haya pasado de 21, el jugador gana si tiene una suma mayor que la del crupier o si este último se pasa de 21. Si ambos tienen la misma suma, se produce un empate.
- Se considera un **Blackjack natural** conseguir la suma de 21 con las dos primeras cartas, es decir, obteniendo un as y una carta de valor 10. Un Blackjack natural gana siempre a un 21 conseguido con más de 2 cartas. Si jugador y crupier obtienen ambos un Blackjack natural, se produce un empate.

2.2.2. Representación formal del problema

De esta manera, una vez definidas las reglas, presentaremos la representación formal utilizada para el problema.

Espacio de estados

Cada observación del entorno nos dará un estado que definirá la situación actual del agente y la idoneidad de las posibles acciones a ejecutar. Definimos nuestro espacio de estados de la siguiente manera: cada estado estará definido por una tripla (suma actual, carta dealer, as utilizable).

- **Suma Actual**: Describe la suma total de la mano actual del jugador. Podrá tener valores entre (2-32).
- **Carta Dealer**: Describe el valor de la carta enseñada por el crupier. Podrá tener valores entre 1 y 10.
- **As Utilizable**: Valor booleando que indica si tenemos un as que estemos usando con su valor de 11. Si en un momento dado tenemos un as utilizable y nos pasamos de 21, este valor pasará a 0 y en la suma actual contaremos el as con su valor 1.

En ***Gymnasium***, por tanto, el espacio de estados estará definido por un objeto *Tuple* tal que *Tuple(Discrete(32), Discrete(11), Discrete(2))*. Esto da lugar un espacio posible de 31 x 10 x 2 = 620 **estados teóricos**. Sin embargo, veremos que en realidad no todos estos estados son accesibles en el juego. Al final, tendremos **un conjunto menor de estados reales** puesto que algunos estados nunca llegarán a ser visitados. Por ejemplo, como asumimos que un as es utilizable siempre que estemos debajo de 21, para un par inicial de cartas si tenemos dos ases, consideraremos que tenemos un valor de 12 (con un as utilizable), por tanto, nunca visitaremos el estado con la suma actual igual a 2. Lo mismo ocurre para la suma inicial igual a 3. Esto provoca que cada estado inicial siempre esté en los valores: (**Suma Actual**: 4-21, **Carta Dealer**: 1-10, **As Utilizable**: 0-1).

Espacio de acciones

El espacio de acciones estrá definido en el rango {0,1} tal que {0: **Stick** (Plantarse), 1: **Hit** (Pedir carta)}.

Recompensas

Para cada acción tomada por el agente, definimos las recompensas de la siguiente manera:

- **-1** si el agente **pierde**.
- **+1** si el agente **gana**.
- **+1.5** si el agente gana con un **Blackjack natural**.
- **0** en cualquier **otro caso**.

Le damos una pequeña **recompensa adicional** al agente en el caso del **Blackjack natural**, para que aprenda más rápidamente a identificar que esta es la mejor situación ganadora posible y que en este caso la mejor acción es plantarse y no pedir más cartas. Además, una vez que decidimos una acción (ya sea plantarnos o pedir carta), el entorno de *Gymnasium* decidirá si el jugador gana o pierde. Es decir, en caso de plantarnos repartirá automáticamente las cartas del crupier para comparar su mano con la del jugador (o ver si se pasa); y en caso de pedir carta, verá si nos pasamos (terminando la ronda) o podemos seguir jugando. Por tanto, el estado que tengamos en ese momento será el que se convierta (o no) en estado terminal. Esto provoca que, a pesar de que podamos usar valores hasta 32 para representar la suma actual del jugador, en realidad nunca llegaremos a visitar estados por encima del valor 21. Por ende, tenemos **un conjunto aún menor de posibles estados reales visitables**.

2.2.3. Contexto y abordaje del problema

Antes de proceder a abordar el problema, debemos estudiar y tener en cuenta antes algunas características propias del Blackjack en general, y de la implementación del mismo en *Gymnasium* en particular. Lo primero que merece ser destacado es que el entorno de *Gymnasium* asume durante la ejecución del problema que estamos usando **una baraja de cartas infinita**. Esto equivale a tener *muestreo con reemplazo* en nuestro experimento, o lo que es lo mismo, **las probabilidades** de sacar cada carta de la baraja **nunca cambian**. Es decir, cada vez que sacamos una carta con un valor concreto (por ejemplo, un as), las probabilidades de sacar otra vez la misma no disminuyen, a diferencia de lo que ocurriría con una baraja real.

Además, al tener todas las figuras un valor de 10 y el as un potencial valor de 11, tenemos un gran número de cartas de alto valor en cada palo: **4 de valor 10 y 1 de valor 11**. Podemos deducir entonces, que la concurrencia de ambos hechos provocará que tengamos muchas probabilidades de tener una mano inicial de cartas con una suma total bastante alta. Si observamos la figura 2.1, podemos ver la distribución obtenida para la suma inicial del jugador en un total de 100.000 episodios.

En esta podemos ver claramente que la mano de cartas más probable que podemos obtener en el estado inicial suma un valor de 20. También podemos ver que la distribución presenta un sesgo negativo, con la mayoría de valores concentrados

images/sumaInicial.png

Figura 2.1: Distribución de suma inicial del jugador para N=100000

más en la parte derecha, donde están los valores altos, que en la izquierda. Tenemos incluso la misma probabilidad de obtener un 21 que un 11. Por tanto, es fácil ver el hecho de que tener manos muy altas al inicio de cada episodio lo que provocará será que tengamos episodios de corta duración. En la mayoría de ocasiones, o bien deberemos plantarnos o nos pasaremos de 21 al pedir otra carta, lo que hará terminar el episodio en ambos casos. Esto limitará, por consecuencia, las posibilidades de aprendizaje del agente puesto que reducirá la experiencia que podrá adquirir en cada episodio: al tener episodios cortos, visitaremos menos estados y actualizaremos menos sus valores en las tablas de cada algoritmo. Además, el Blackjack presenta otro problema fundamental: al ser un juego de azar, la visita de muchos estados dependerá de las cartas aleatorias que le toquen al jugador y al crupier. Esto supondrá que muchas veces nos costará visitar muchos estados del problema. Para contrarrestar esto, tendremos que ejecutar un número de episodios suficientemente alto para que los algoritmos puedan converger a una solución. **MARGEN de mejora pequeñam, óptima recompensa y duración promedio**

2.2.4. Trabajos relacionados y aportes originales

Este problema es ampliamente conocido dentro del ámbito del aprendizaje por refuerzo. Sobre todo, por ser tratado por Sutton y Barto en su libro [8]. Además, se pueden encontrar diversas aproximaciones y estudios online al respecto [4][1]. Nuestra mayor aportación a este problema es que nosotros en vez de centrarnos en estudiar los valores de las tablas Q, hemos intentado hacer un profundo estudio del problema para intentar resolverlo así como algunos análisis más originales. Para abordar este problema, realizaremos nuestro análisis comparativo usando el entorno de Blackjack de *Gymnasium* entrenando a los agentes durante 100.000 episodios. Para el análisis final haremos los entrenamiento 10 veces con semillas distintas. Tras el entrenamiento de un agente que toma decisiones aleatorias que usaremos como rendimiento baseline, haremos un análisis detallado de los parámetros de los algoritmos. Además, haremos experimento con versiones de los mismos con ϵ-decaimiento así como un análisis en detalle de la cobertura de estados que hacen los mismos.

2.3. Algoritmos

Para tratar el problema, hemos creado agentes con algoritmos basados en métodos de Monte Carlo y algoritmos basados en Diferencias Temporales.

2.3.1. Métodos de Monte Carlo

Implementamos los métodos de Monte Carlo On-policy y Off-policy. Estamos usando la variante de evaluación en todas las visitas (Every-visit). Además, usamos en ambos políticas ϵ-*soft*, en concreto ϵ-*greedy* para crear las muestras de los episodios.

Los pasos básicos que realizan son los siguientes:

1. Se generan los datos del episodio usando una política determinada.
2. Se itera el episodio hacia atrás, calculando el retorno acumulado con el factor de descuento
3. Se actualiza el valor estado-acción de las tablas Q(s,a) usando el retorno y ponderado por la tasa de aprendizaje.

Podemos ver la implementación de la versión On-Policy en**Algorithm 4**.

Algorithm 4 Monte Carlo On-Policy (Every-visit)

Parámetro: $\epsilon > 0$ pequeño
Inicializar:
$\pi \leftarrow$ política ϵ-soft arbitraria
$Q(s,a) \in \mathbb{R}$ arbitrariamente, $\forall s \in S, a \in A(s)$
$Returns(s,a) \leftarrow$ lista vacía, $\forall s \in S, a \in A(s)$
while verdadero **do** ▷ Para cada episodio
  Generar un episodio siguiendo π:

$$S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T$$

  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    Agregar G a $Returns(S_t, A_t)$
    $Q(S_t, A_t) \leftarrow promedio(Returns(S_t, A_t))$
    $A^* \leftarrow \arg\max_a Q(S_t, a)$
    for cada $a \in A(S_t)$ **do**
      if $a = A^*$ **then**
        $\pi(a|S_t) \leftarrow 1 - \epsilon + \frac{\epsilon}{|A(S_t)|}$
      else
        $\pi(a|S_t) \leftarrow \frac{\epsilon}{|A(S_t)|}$
      end if
    end for
  end for
end while

La principal diferencia de la versión Off-Policy del algoritmo es que, como ahora usamos una política distinta de la objetivo para generar las muestras, a la hora de evaluar y actualizar los estados usamos una media ponderada incremental. Quedando la línea de actualización

$$Q(s,a) \leftarrow Q(s,a) + \frac{W}{C(s,a)} * (G - Q(s,a))$$

El algoritmo quedaría entonces como **Algorithm 5**

Algorithm 5 Monte Carlo Off-Policy con muestreo por importancia ponderado

Inicializar $Q(s,a)$ arbitrariamente
Inicializar $C(s,a) \leftarrow 0$
$\pi(s) \leftarrow \arg\max_a Q(s,a)$
while verdadero **do**
  Generar episodio usando política de comportamiento μ
$$S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$$
  $G \leftarrow 0$
  $W \leftarrow 1$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}(G - Q(S_t, A_t))$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    if $A_t \neq \pi(S_t)$ **then**
      break
    end if
    $W \leftarrow W \cdot \frac{1}{\mu(A_t|S_t)}$
  end for
end while

2.3.2. Métodos de Diferencias Temporales

Hemos implementado en los agentes basados en métodos de Diferencias Temporales los algoritmos Sarsa y Q-learning. Ambos en su versión de un paso. El esquema principal que siguen estos es:

Los pasos básicos que realizan son los siguientes:

1. Se generan los datos usando una política determinada.
2. Para cada paso de cada episodio, se observa la recompensa obtenida y el nuevo estado tras ejecutar una acción según la política anterior.
3. Se actualiza el valor estado-acción de las tablas Q(s,a) usando la recompensa inmediata y el valor estimado del nuevo estado al que se llega, haciendo así *bootstrapping*.
4. Se repite esto hasta llegar al final del episodio.

El algoritmo SARSA quedaría como **Algorithm** 6:

La única diferencia de Q-learning respecto a SARSA es que ahora el algoritmo, por ser Off-policy, selecciona el máximo de los valores estado-acción, quedando la línea de actulización de la siguiente manera:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

2.4. Evaluación/Experimentos

- Configuración experimental: herramientas, entornos de prueba, datasets empleados, etc.
- Métricas utilizadas para evaluar el desempeño del método propuesto.
- Resultados obtenidos, presentados en tablas o gráficos según corresponda.

Algorithm 6 SARSA

Parámetros: tamaño de paso $\alpha \in (0,1)$, ϵ pequeño > 0
Inicializar $Q(s,a)$ arbitrariamente para todo $s \in S^+$ y $a \in A(s)$
for cada episodio **do**
  Inicializar estado S
  Elegir acción A desde S usando política derivada de Q (por ejemplo, ϵ-greedy)
  while S no sea terminal **do**
    Ejecutar acción A
    Observar recompensa R y nuevo estado S'
    Elegir acción A' desde S' usando política derivada de Q (ϵ-greedy)
    $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$
    $S \leftarrow S'$
    $A \leftarrow A'$
  end while
end for

- Análisis crítico de los resultados y comparación con otros enfoques.

Como ya hemos indicado, todos los experimentos del análisis comparativo han sido realizados mediante el entorno de Blackjack mediante la librería *Gymnasium* enfocada al Aprendizaje por Refuerzo. A continuación, comentaremos los resultados más relevantes tras la experimentación.

2.4.1. Rendimiento baseline: Agente aleatorio

Lo primero que hicimos fue diseñar un primer experimento que nos permitiera obtener una medida de rendimiento de línea base (o *baseline*). De esta manera, intentaríamos después mejorar este rendimiento base con los algoritmos principales de aprendizaje entrenados. Para ello, construimos un primer agente aleatorio: cada acción que elige para cada estado del agente es tomada totalmente al azar. En la figura 2.2 podemos ver los resultados tras entrenar al agente durante 10.000 episodios en 5 ejecuciones (con 5 semillas distintas).

`images/agenteRandom.png`

Figura 2.2: Rendimiento de agente aleatorio durante 5 ejecuciones

Podemos ver, como era de esperar, que los resultados del agente se muestran constantes a lo largo del tiempo, no existe mejora. A lo largo del aprendizaje obtenemos una recompensa media por episodio de -0.4 y una longitud media de episodio de 1.4 acciones. Como ya habíamos comentado antes, esto nos da un margen de mejora pequeño para el resto de agentes, especialmente en cuanto a longitud de episodio se refiere.

2.4.2. Influencia de parámetros

Antes de realizar el entrenamiento final de cada agente, realizamos un estudio y análisis de parámetros para cada uno de los agentes construidos. De esta manera, intentamos encontrar la mejor configuración para los parámetros $\alpha, \epsilon y \gamma$ de cada algoritmo así como comprobar cuál de ellos es más influyente en el rendimiento del mismo. El *grid* de parámetros utilizado para cada uno fue:

- **Alfa** (tasa de aprendizaje/tamaño del paso): Valores fijos de 0.05 y 0.01 y valor variable de $1/N(S_t)$.
- **Gamma** (factor de descuento): Valores de 0.8, 0.95 y 1.0.
- **Épsilon** (parámetro de exploración): Valores de 0.01, 0.1 y 0.2.

Cada agente fue entrenado con esta parrilla de parámetros durante 20.000 episodios para 5 ejecuciones con semillas distintas. A continuación, en la figura se muestran los resultados para el agente Monte Carlo On-policy, pero todos mostraron comportamientos muy similares. Tras el estudio de la influencia de los parámetros en los agentes, en general, las conclusiones obtenidas fueron las siguientes:

- **Alfa**: Dado un número fijo de episodios, valores relativamente medianos/altos han demostrado ser más efectivos para la tasa de aprendizaje. Esto tiene sentido, puesto que al tener muchos estados que se visitan poco o muy poco (como ya veremos más adelante), un mayor tamaño de actualización nos permitirá un aprendizaje más rápido. Por ello, elegimos un **valor de 0.05** para el entrenamiento de nuestros agentes. Sin embargo, suponemos que para un número indefinido de episodios (tendiendo a infinito), el valor variable de $1/N(S_t)$ será el que permita a los agentes acercarse más al valor óptimo (sobre todo a los basados en diferencias temporales), puesto que es el único que respeta las condiciones de aproximación estocástica.
- **Épsilon**: Para el parámetro de exploración, valores más pequeños han demostrado ser mucho más efectivos. Como ya veremos más adelante, el problema de que muchos estados del Blackjack se visiten poco no se debe al factor de exploración. Este parámetro ha probado ser el que más repercute en el rendimiento de los agentes. Por esto, elegimos un valor de **0.01**.
- **Gamma**: Para un número de 20.000 episodios, el factor de descuento ha resultado tener cierta influencia en el rendimiento de los agentes. A priori, cabría esperar que, en un problema como este en el que tenemos una duración media de episodios de entre 1 y 2 acciones, y además sólo obtenemos recompensa al final de cada episodio, el factor de descuento no fuese demasiado determinante. La mayoría de episodios terminan con una acción y por tanto, no aplican el factor de descuento. Y en las pocas ocasiones en que sea aplicado al tener episodios de 2 o más acciones, la diferencia de retornos debería ser pequeña (los episodios de más de 3 acciones son estadísticamente raros).

  Sin embargo, la influencia que parece tener el factor gamma en las gráficas probablemente se deba al hecho de que, en el Blackjack, con sólo 20000 episodios aún tenemos muchos estados que se han visitado muy poco. En esta etapa el agente aún se encuentra en la fase de aprendizaje inicial, aún no ha llegado a la fase de convergencia. Por tanto, en este caso, los parámetros gamma con valores más pequeños probablemente estén actuando como factor de regularización para datos ruidosos (especialmente en los valores de los estados poco visitados), tanto de los retornos de Monte Carlo como de los valores estimados en *Bootstrapping*. Es esperable, tal y como se muestra en la figura 2.4, que al ser entrenado con muchos más episodios, las diferencias causadas por el factor de descuento en el rendimiento de los agentes se vayan disipando. Por ello, como para el aprendizaje final emplearemos de los mismos usaremos 100.000 episodios, tomaremos **1** como valor de gamma para simplificar los cálculos.

images/sens1.png

(a) Monte Carlo On-Policy

images/sens2.png

(b) Monte Carlo Off-Policy

images/sens3.png

(c) SARSA

images/sens4.png

(d) Q-learning

Figura 2.3: Influencia de parámetros en recompensa media. - γ-α-ϵ de izquierda a derecha.

images/sens5.png

Figura 2.4: Influencia de parámetros en recompensa media en MC On-Pol para 50000 episodios - γ-α-ϵ de izquierda a derecha.

2.4.3. Experimentos con ϵ-decaimiento

Para intentar paliar el problema antes mencionado de la cobertura de estados, procedimos a implementar también mecanismos de ϵ-decaimiento en nuestros agentes. Hicimos el entrenamiento de los agentes, tanto con los valores de épsilon escogidos en la sección anterior, como con un valor de 0.1 que fuese disminuyendo usando decaimiento lineal hasta alcanzar su valor predeterminado (de 0.01) durante la mitad del aprendizaje.

Tras realizar la prueba, pudimos comprobar que en ningún caso tener un mayor valor de ϵ al principio del aprendizaje nos proporcionó un mejor rendimiento de los agentes. Ni siquiera en aquellos que usan aprendizaje On-Policy (MC On-Pol y SARSA) como puede verse en la figura 2.5, que por su propia naturaleza tienen un aprendizaje más lento por ser menos exploratorios. Esto nos desmuestra que el hecho de que tengamos estados que sean muy poco visitados no se debe al factor de exploración, sino que se deben a otros motivos, tal y como

images/decay1.png

Figura 2.5: Rendimiento y duración medios por episodios para MC y SARSA (sin y con decaimiento).

analizaremos en la siguiente sección.

2.4.4. Análisis de la cobertura de estados

El aprendizaje de los agentes en el problema de Blackjack se ve limitado por un problema fundamental: **hay algunos pocos estados que son muy visitados durante en el entrenamiento y muchos estados que son visitados muy poco**. Esto puede verse perfectamente en la figura 2.6. En ella podemos ver que la distribución de visitas por estado tiene un claro sesgo positivo: la mayor parte de datos están a la izquierda. Esto implica que, para un entrenamiento de 100.000 episodios, la mayor parte de estados son visitados menos de 1000 veces; mientras que luego, un pequeño número de estados, son visitados varios miles de veces. Además, vemos cómo los tres agentes visitan el mismo número de estados (280/620). Si eliminamos de los 620 estados teóricos aquellos con suma igual a 1,2,3 y superior a 21, puesto que como ya vimos estos nunca llegan a ser visitados, estos número de estados parece verosímil como posible número de **estados reales visitables**.

images/coberMCon.png

(a) Monte Carlo On-Policy

images/coberMCoff.png

(b) Monte Carlo Off-policy

images/coberQ.png

(c) Q-learning

Figura 2.6: Cobertura de estados para varios agentes

En un principio, pudimos achacar estos resultados a que los algoritmos no eran lo suficientemente exploratorios. Sin embargo, como ya hemos visto en los análisis anteriores, ni usar mayores tamaños fijos de ϵ ni usar un tamaño mayor variable con decaimiento, no permitió obtener mejoras de rendimiento para ningún agente. Además, si se comparan los resultados de la cobertura de Monte Carlo On-Policy con otros más exploratorios (agentes Off-Policy como MC Off-Pol y Q-learning), podemos ver que son totalmente similares. Por tanto, podemos concluir que el factor de exploración ϵ no es lo que está limitando las visitas de los estados poco visitados.

El problema en realidad es otro: **el Blackjack, por ser un juego de azar, tiene una alta varianza estocástica.** Para ver esto, analizamos la frecuencia con la que se visitan los distintos estados en cada agente. En este análisis (y los siguientes), nos centraremos únicamente en los estados que el jugador tiene una suma actual mayor que 11, tal y como hacían Sutton y Barto en [8]. El motivo es que estos son los verdaderamente interesantes de analizar, puesto que es fácil ver que, con cualquier mano inferior a 12, la jugada óptima siempre debe ser pedir carta. Por ello, nos centraremos en analizar los estados que resultan claves para ganar la partida, especialmente aquellos que se encuentran en la frontera crítica entre victoria y derrota (los que están en la suma entre 15-17), puesto que en ellos no es fácil determinar cuál es la mejor acción que tomar. Además, desambiguaremos los estados según tengamos o no un as utilizable.

images/estadosMCon.png

(a) Monte Carlo On-Policy

images/estadosQlear.png

(b) Q-learning

Figura 2.7: Frecuencia de visitas sobre los estados de MC On-Pol y Q-learning

Podemos ver en la figura 2.7 que para dos agentes con funcionamientos totalmente distintos como MC On-policy y Q-learning la frecuencia con la que visitan los estados del juego es prácticamente indistinguible entre ambos. Lo que nos está limitando, por tanto, es la alta variabilidad que existe al repartir las cartas y las probabilidades que tiene cada mano de tocarle tanto al jugador como al crupier. Es la propia aleatoriedad al repartir cartas de la baraja la que nos dejará visitar los estados con más o menos frecuencia. En la figura 2.7 se ve claramente que, en ambos casos, el estado más visitado cuando no tenemos un as utilizable es aquel en el que el jugador tiene un 20 y el crupier enseña un 10. Como ya comentamos

anteriormente, esto cuadra perfectamente con los resultados analizados en la sección 2, puesto que tenemos varias cartas con valor 10 y la mano inicial más probable al jugar es 20. De hecho, podemos ver perfectamente que los estados de la columna con valor 10 para el crupier son mucho más visitados que el resto. Por otra parte, cuando el jugador tiene un as utilizable, el estado más visitado sin duda es aquel en el que este tiene un 21 (es decir, un as más un 10) y el crupier un 10. Vemos también que la fila y la columna de este mismo estado son las que más son visitadas con diferencia, por las mismas razones explicadas. Merece destacar además, si nos fijamos en la leyenda, que los estados de las tablas con as utilizable reciben muchas menos visitas totales. Esto se debe, obviamente, a que para acceder a estos estados debemos tener la suerte de que al jugador le toque un as, que tiene una probabilidad de 1/13 de aparecer en la baraja.

Por tanto, como la principal limitación del aprendizaje de los agentes es la varianza estocástica propia del juego de azar, lo que hicimos para contrarrestarlo fue entrenarlos con un número de episodios lo suficientemente alto. Por ello, **escogimos usar 100.000 episodios**.

2.4.5. Comparativa global

Por último, vemos los resultados globales de los rendimientos de los agentes. Recordemos que cada uno de ellos ha sido entrenado durante 100.000 episodios en 10 ejecuciones con distinta semilla.

`images/global.png`

Figura 2.8: Recompensa y duración promedios por episodio para los agentes

Si observamos la gráfica 2.8, podemos ver que los 4 algoritmos terminan convergiendo en una recompensa media de entre -0.04 y -0.05 por episodio. Como ya explicamos antes, en el Blackjack no es posible alcanzar una media de 0, el casino siempre tiene una pequeña ventaja. Sin embargo, esto no quiere decir que todos estén funcionando igual de bien. Para encontrar las claras diferencias en el rendimiento de los algoritmos, debemos atender en su lugar a la duración media por episodio. Una mayor duración por episodio indica que el agente está aprendiendo mejor el funcionamiento interno del juego, aunque al final por el azar del mismo obtenga recompensas parecidas, puesto que es capaz de jugar más manos sin pasarse de 21. En la figura podemos ver claramente que algunos agentes superan claramente a otros en el aprendizaje del Blackjack.

Podemos ver que Monte Carlo Off-policy, por su naturaleza más exploratoria, tiene un aprendizaje mucho más rápido al principio que la variante On-Policy. Sin embargo, como era de esperar, ambos terminan convergiendo sobre una duración media de 1.5 acciones por episodio. Por otro lado, los métodos basados en Diferencias Temporales superan claramente a los de Monte Carlo. Vemos que SARSA consigue obtener una duración media cercana a 1.55 mientras que Q-learning, que es claramente el ganador, llega a alcanzar una media de 1.6 acciones por episodio. Como ya estudiamos en clase, los métodos de Diferencias Temporales consiguen ajustarse de manera más verosímil al proceso de decisión de Markov subyacente en el problema, como es este caso. Los métodos de Monte Carlo se ajustaban mejor a los datos obtenidos durante los episodios. Podemos deducir, por tanto, que el bootstrapping está ayudando a los algoritmos a aprender mejor el verdadero valor de los estados en un problema como este con episodios muy cortos. Creemos que esto ha sido especialmente útil para aquellos estados que son raramente visitados, puesto que **al actualizarlos mediante bootstrapping, la propagación del valor de estos es más rápida que en Monte Carlo** que necesita varios episodios completos para mejorar los valores de los estados más cercanos al inicio. **Esto es fundamental en un problema como este con episodios especialmente cortos y con muchos estados visitados muy raramente**.

Si comparamos los métodos On-Policy (MC On-Pol y SARSA) con los Off-Policy (MC Off-Pol y Q-learning), vemos que los primeros tienen un aprendizaje más lento pero algo más estable mientras que los últimos consiguen un aprendizaje más rápido pero algo más ruidoso (se vería más claramente si hiciese *zoom* sobre las curvas).

Por último, podemos ver que claramente **todos los agentes entrenados han conseguido superar el rendimiento *baseline*** que nos marcamos, puesto que el agente aleatorio conseguia una recompensia promedio de -0.4 y duración media por episodio de 1.4.

2.5. Conclusiones

2.5.1. Conclusiones y resumen del trabajo

Tras el estudio del problema, la realización de los experimento y el análisis comparativo de los algoritmos, las principales conclusiones obtenidas han sido las siguientes:

- El Blackjack resulta un **juego bastante difícil de aprender** para los agentes. El poco margen de mejora en las recompensas que obtenemos, puesto que nunca podremos tener las mismas victorias o más que el crupier, junto con el tamaño tan reducido de los episodios hace que el aprendizaje se vea más limitado.
- Un agente que decide de manera totalmente aleatoria obtiene una recompensa media por episodio de -0.4 y una duración media de episodios de 1.4.
- Es fundamental hacer una correcta selección de parámetros para los algoritmos. **El factor de exploración ϵ ha demostrado ser el que más influencia tenga** sobre el rendimiento de los agentes. Valores más pequeños han probado ser mejores. Elegimos un valor de 0.01. El siguiente en influencia sobre los resultados resultó ser la tasa de aprendizaje α. Valores fijos como 0.05 han demostrado ser mejores que valores variables como $1/N(S_t)$ que

a la larga se harán más pequeños que el primero cuantos más episodios tengamos. Esto se debe a que es mejor tener un mayor tamaño de actualización si tenemos un problema con episodios muy cortos y muchos estados que se visitan poco. El parámetro que menos influye a la larga sobre el rendimiento de los algoritmos resultó ser el factor de descuento γ. Esto tiene sentido puesto que estamos tratando con episodios muy cortos donde este apenas tiene repercusión. Por simplificar, elegimos un valor de 1.

- Usar un mayor valor de ϵ e incluyendo **decaimiento lineal no mejoró el rendimiento de los algoritmos**. Al contrario, lo empeoró. Por tanto el problema del lento aprendizaje del Blackjack no es el factor de exploración.
- En el Blackjack **hay algunos pocos estados que son muy visitados durante en el entrenamiento y muchos estados que son visitados muy poco**. Además, todo los agentes hacen una cobertura muy similar de los estados visitables. Tanto los métodos de Monte Carlo como los basados en Diferencias Temporales visitan los estados con prácticamente la misma frecuencia. La clave está en que el el Blackjack, por ser un juego de azar, tiene una alta varianza estocástica. La aleatoriedad de las cartas repartidas será la que decida realmente cómo nos movemos por el espacio de estados y qué estados visitaremos más.
- Por este problema de alta varianza estocástica, **los agentes necesitan muestrear muchos episodios para realizar un correcto entrenamiento**. Elegimos entrenarlos durante 100.000 episodios.
- Tras el análisis final todos los agentes han **convergido en una recompensa media por episodio de -0.04**. Sin embargo, **algunos han aprendido a jugar al juego mejor que otros** puesto que han sido capaces de generar episodios más largos (y por tanto, jugar más manos) a lo largo del entrenamiento. Monte Carlo On-Policy ha sido el que peores resultados ha dado con una duración media de 1.5 acciones por episodio. Q-learning ha sido el mejor de todos con una una duración media de 1.6.
- **Los métodos basados en Diferencias Temporales han demostrado ser más efectivos en el aprendizaje que los métodos de Monte Carlo**. Estos consiguen adaptarse mejor a la dinámica intrínseca del juego. El bootstrapping está ayudando a los algoritmos a aprender mejor el verdadero valor de los estados en un problema con episodios muy cortos. Especialmente en los estados que son raramente visitados, puesto que **al actualizarlos mediante bootstrapping, la propagación del valor de estos es más rápida que en Monte Carlo** que necesita varios episodios completos para mejorar los valores de los estados más cercanos al inicio. Esto es fundamental en un problema como este con episodios especialmente cortos y con muchos estados visitados muy raramente.
- Los métodos On-Policy (MC On-Pol y SARSA) tiene un aprendizaje más lento pero algo más estable mientras que los métodos Off-Policy (MC Off-Pol y Q-learning), consiguen un aprendizaje más rápido pero algo más ruidoso.
- Todos los agentes han conseguido superar el rendimiento *baseline* del agente aleatorio.

2.5.2. Limitaciones y vías futuras

Como ya hemos visto, la principal dificultad que han tenido los agentes para no alcanzar la política óptima ha sido la limitación inherente que tiene el problema como juego de azar. Al depender de la aleatoriedad de las cartas para poder visitar muchos de los estados, para poder actualizar sus valores en la dirección correcta necesitas tener un número de episodios lo suficientemente grande. Los métodos basados en Diferencias Temporales han demostrado ser más efectivos en el aprendizaje que los métodos de Monte Carlo.

Como posible vías futuras de trabajo sería interesante ver si aumentando el número de episodios a 500.000 o a más de un millón como en otros estudios [1][8], podríamos ser capaces de alcanzar la política óptima. Nos habría gustado añadir en este estudio, además, un análisis estructural detallado de cómo se van actualizando las tablas de los valores Q(s,a) en los distintos agentes y las diferencias entre ellos a la hora de hacerlo. También hubiese sido interesante ver cómo actúa el sesgo de maximización en los métodos de Diferencias Temporales analizando si SARSA aprende a jugar al Blackjack de manera más conservadora y si Q-learning lo hace de forma más arriesgada, intentando maximizar las recompensas en jugadas de mayor riesgo.

Capítulo 3

Aprendizaje en Entornos Complejos

3.1. Introducción

Flappy Bird es un entorno de Gymnasium donde un agente debe controlar a un pájaro para navegar a través de huecos entre tuberías[3]. Es un problema de aprendizaje por refuerzo episódico con espacios de estados cotinuos (posición horizontal/vertical de las tuberías, velocidad, rotación) y un espacio de acciones discreto binario (saltar o no saltar). Es una tarea de control interesante por su baja tolerancia al error, un error puede significar la terminación y pérdida del juego.

La importancia de este problema se basa en su baja tolerancia al error, permite estudiar el efecto de estrategias on-policy que pueden aprender políticas más conservadoras y estrategias off-policy que podrían arriesgar más en busca de maximizar la recompensa, este problema podría plantear una analogía con el problema 'Cliff Walking' descrito por [8].

Se pretende implementar y calibrar los algoritmos SARSA Semi gradiente y Deep Q-learning, utilizando para el primero Tile coding para aproximar las funciones dado el espacio de estados continuos. Además se pretende analizar el impacto de diferentes arquitecturas en Deep Q-learning, en soluciones basadas en n-pasos y en agrupación de estados o información de varios estados por ejemplo los 3 últimos estados. Analizaremos la velocidad de convergencia, la estabilidad del aprendizaje, estimaremos el riesgo asumido por cada algoritmo y trataremos de visualizar la política aprendida con mapas de calor enfatizando 2 características del estado.

El documento se organiza en una sección de desarrollo que define el problema, los enfoques existentes y los antecedentes, explicaremos los métodos utilizados para abordar el problema y justificaremos en que se diferencia el trabajo actual con respecto a los anteriores.

3.2. Desarrollo

3.2.1. Descripción del problema

El juego de Flappy Bird es un entorno de control de precisión donde el objetivo primordial es navegar a través de una serie de tuberías con huecos variables, manteniendo al pájaro en vuelo el mayor tiempo posible y evitando colisionar con los obstáculos o los límites del escenario. Se define como una tarea episódica de aprendizaje por refuerzo en la que un solo error conduce a un estado terminal inmediato. Las normas fundamentales que rigen el comportamiento del entorno son:

- Al inicio de cada episodio, el agente (el pájaro) comienza en una posición determinada sujeto a una fuerza de gravedad constante que debe contrarrestar para no caer
- En cada frame, el agente dispone de un espacio de acciones binario: puede elegir entre no hacer nada (0), permitiendo que la gravedad actúe, o saltar (1), aplicando un impulso ascendente que modifica su trayectoria.
- El agente percibe el estado mediante información numérica continua (Opción 2), que incluye las posiciones de las tuberías próximas, así como la posición vertical, velocidad y rotación del pájaro.
- El objetivo se formaliza mediante recompensas: se otorgan $+0.1$ puntos por cada frame de supervivencia y $+1.0$ puntos por superar cada tubería. Por el contrario, se penaliza con -1.0 al morir por colisión y -0.5 al tocar el límite superior de la pantalla.
- El episodio termina instantáneamente si el pájaro choca contra una tubería o el suelo.

3.2.2. Representación formal del problema

Una vez definidas las reglas, presentaremos la representación formal utilizada para el problema de Flappy Bird basado en la Opción 2 de observaciones[3].

Espacio de estados

Cada observación del entorno nos dará un estado que define la situación cinemática del agente y los obstáculos inmediatos. Definimos nuestro espacio de estados de la siguiente manera,cada estado es un vector de 12 variables continuas:

- Posiciones de tuberías (9 variables): Incluye la posición horizontal y las posiciones verticales (superior e inferior) de la tubería anterior, la siguiente y la subsiguiente.
- Estado del jugador (3 variables): Describe la posición vertical, la velocidad vertical y la rotación actual del pájaro.

En Gymnasium, este espacio está definido por un objeto Box de 12 dimensiones: Box(low=-inf, high=inf, shape=(12,), dtype=np.float32). Aunque el espacio es infinito, la dinámica del juego restringe los estados accesibles. Por ejemplo, la rotación y la velocidad están limitadas por la física del entorno,el pájaro no puede visitar posiciones fuera de los límites de la pantalla sin terminar el episodio, no puede tener una rotación que lo coloque mirando hacia el inicio, etc. Las variables que modelizan el estado son números en coma flotante por lo que potencialmente podríamos tener 12^{2^b} estados diferentes, siendo b el número de bits utilizados para representar esos

números(en este caso 32), es un conjunto que no podría ser almacenado por un ordenador, impidiendo el uso de métodos tabulares.

Espacio de acciones

El espacio de acciones discreto está definido en el rango 0,1 tal que:

- 0: No hacer nada. El pájaro cae por efecto de la gravedad.
- 1: Saltar (flap). Se aplica un impulso ascendente al agente.

Recompensas

Para cada acción tomada por el agente, definimos las recompensas de la siguiente manera:

- +0.1 por cada frame que el agente permanezca con vida.
- +1.0 por cada tubería superada con éxito.+1.0 por cada tubería superada con éxito.
- -1.0 si el agente colisiona (estado terminal).
- -0.5 si el agente toca el límite superior de la pantalla.

Siguiendo la hipótesis de la recompensa, estos valores buscan formalizar el objetivo de supervivencia a largo plazo. La penalización por tocar el techo incentiva al agente a mantenerse en zonas seguras, evitando trayectorias extremas.

3.2.3. Trabajos Relacionados

Hay varias representaciones del problema, el entorno que nosotros usamos da dos opciones, pero hay otros entornos como [5]. Teniendo eso en cuenta en la literatura han abordado el problema con entornos gráficos donde los estados son las imágenes y por tanto han usado redes convolucionales [6], tambien se ha utilizado la aproximación de LiDAR con un estimador Q basado en transformers [2]. También se han comparado para el problema arquitecturas como SARSA y Q-Learning [9] consiguiendo resultados bastante buenos con puntuaciones por encima de 1400. En muchos de estos estudios utilizan Double Q-learning con una red "target"para mitigar el problema de machacar los pesos aprendidos en cada iteración.

3.2.4. Aportes del trabajo

Nuestro trabajo aporta una idea basada en la agregación de estados, como por ejemplo en la memoria episódica que usan en [2], con un buffer de estados anteriores que se usan como entrada de un transformer. A su vez en este estudio se basaron en el "frame stacking-[7]. Ideas parecidas también son mencionadas en el libro [8] donde dicen que un estado puede estar formado por una sensación inmediata, transformaciones complejas de sensaciones inmediatas o estructuras construidas atrevas del tiempo en secuencias de sensaciones. Nosotros estudiaremos el resultado de crear un entorno formado por k instantes de tiempo para tomar la siguiente acción, esto como contraparte genera una mayor complejidad del conjunto de estados posibles porque ahora un estado podría estar representado por k secuencias de observaciones si antes teníamos 12 parámetros por estados ahora tendíamos 12*k. Una estrategia basada en redes neuronales no debería de tener demasiados problemas generalizando los estados similares, incluso puede aprender otras componentes derivadas de usar más estados como por ejemplo inferir la aceleración. Mientras que una estrategia lineal como SARSA semi gradiente si que podría padecer de la maldición de la dimensionalidad al hacer un problema complejo aún más complejo.

3.3. Algoritmos

3.3.1. Baseline aleatorio

Como línea base, se evalúa un agente sin aprendizaje que selecciona acciones de manera uniforme al azar. Este estimador permite verificar que el entorno no se resuelve por azar y sirve como referencia mínima de rendimiento.

$$\pi(a|s) = \frac{1}{|\mathcal{A}|}, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$$

donde:

- $\pi(a|s)$ es la probabilidad de elegir la acción (a) estando en el estado (s).
- $\mathcal{A}$ es el número total de acciones posibles.
- Para cualquier estado (s), todas las acciones tienen la misma probabilidad.

.

Pasos del algoritmo:

- Inicializar el entorno y semilla.
- Para cada episodio, repetir hasta terminar:
  - Elegir acción aleatoria.
  - Ejecutar acción y observar recompensa y transición.
  - Acumular recompensa, número de pasos y puntuación.
- Devolver métricas agregadas (recompensa, pasos, puntuación).

3.3.2. SARSA semi-gradiente (on-policy) con Tile Coding

Se implementa SARSA semi-gradiente con aproximación lineal:

$$\hat{Q}(s, a; \mathbf{w}) = \mathbf{w}_a^\top \mathbf{x}(s)$$

donde $x(s)$ es un vector de características generado por Tile Coding.

Actualización (n-step, en nuestro caso principalmente (n=1):

$$\mathbf{w}a_t \leftarrow \mathbf{w}a_t + \alpha \left(G_t^{(n)} - \hat{Q}(s_t, a_t) \right) \mathbf{x}(s_t)$$

donde:

- $\mathbf{w}_{a_t}$: vector de pesos asociado a la acción (a_t).
- α: tasa de aprendizaje (learning rate), controla el tamaño del cambio.
- $G_t^{(n)}$: retorno objetivo n-step desde el tiempo (t) (recompensas acumuladas en (n) pasos).
- $\hat{Q}(s_t, a_t)$: valor Q estimado actual para el estado (s_t) y acción (a_t).
- $\left(G_t^{(n)} - \hat{Q}(s_t, a_t) \right)$: error temporal.

- $\mathbf{x}(s_t)$: vector de características del estado (s_t) salida de tile coding.

Pasos del algoritmo:

- Construir codificador de estados (Tile Coding).
- Elegir acción con política ϵ-greedy sobre $\hat{Q}$
- Guardar transición y calcular retorno n-step.
- Aplicar actualización semi-gradiente sobre el vector de pesos de la acción ejecutada.
- Reducir ϵ por episodio.

Este algoritmo aprende de la acción elegida (on-policy) y no sobre la posible acción teórica que maximice la recompensa en este paso.

3.3.3. Deep Q-Learning

En el proyecto se usa una variante DQN con opción doble, donde una red selecciona la acción y la red objetivo la evalúa. Esto reduce el sesgo de sobreestimación de Q-learning. Cada cierto tiempo se actualizan los pesos de la red objetivo con los pesos de la red que selecciona las acciones, acontinuación se muestra la ecuación de bellman del agente:

$$Q(s, a; \theta) = r + \gamma^n Q(s', argmax_{a'} Q(s', a'; \theta); \theta')$$

donde:

- $Q(s, a; \theta)$: valor Q que predice la red activa con parámetros (θ) para el estado actual (s) y acción (a).
- (r): recompensa observada tras ejecutar (a) en (s).
- γ: factor de descuento ($0 < \gamma \leq 1$); pondera cuánto importan recompensas futuras.
- n: número de pasos del retorno n-step. Por eso aparece γ^n.
- s': siguiente estado (o estado tras n pasos).
- $\arg\max_{a'} Q(s', a'; \theta)$: acción que la red activa considera mejor en (s')).
- $Q(\cdot; \theta'))$: evaluación de esa acción con la red objetivo (parámetros (θ'), más estable).

Pasos del algoritmo:

- Crear dos redes con la misma arquitectura, una red **activa** (Q_θ), que aprende en cada actualización, y una red **objetivo** (Q_{θ^-}), que se mantiene más estable.
- Jugar episodios en el entorno con política (ϵ)-greedy: la mayor parte del tiempo se elige la mejor acción según (Q_θ), y a veces una acción aleatoria para explorar. Cada transición ((s,a,r,s',a')) se guarda en el *replay buffer*.
- Cada cierto número de pasos (`train_freq`): tomar un minibatch aleatorio del buffer, calcular objetivos (y) y ajustar (Q_θ) minimizando el error MSE entre la predicción actual y esos objetivos.
- De forma periódica, copiar los pesos de la red activa en la red objetivo para mantener el aprendizaje estable y evitar oscilaciones fuertes.

3.3.4. Agregación de estados

El wrapper FrameStack de gymnasium, agrega contexto temporal concatenando las últimas (k) observaciones en un único estado extendido. En lugar de trabajar con un estado instantáneo (o_t), el agente recibe:

$$\tilde{s}t = [st - (k+1), s_{t-(k+2)}, \ldots, s_t]$$

donde:

- $\tilde{s}t$: es el estado agregado en el instante k
- s_t: es el último estado observado
- $st - (k+1)$: es el estado observado hace k-1 instantes
- $s_{t-(k+2)}$: es el estado observado hace k-2 instantes

Esto permite capturar la dinámica del agente (tendencia, aceleración o cambios recientes).

- Inicialización: se define (k) y se crea una cola (deque) de tamaño máximo (k).
- Espacio de observación: se amplía repitiendo los límites low/high del entorno (k) veces, para reflejar la concatenación temporal.
- Reset: al comenzar un episodio, la primera observación se replica (k) veces para evitar estados incompletos.
- En cada paso: se añade la observación nueva a la cola y se devuelve la concatenación de las k observaciones almacenadas.

Esta implementación tambian añade una dimensionalidad mayor en un factor de k, incrementa memoria y tiempo de ejecución en algunos casos, pero puede aportar información relevante al agente sobre todo cuando se usa Deep Learning para estimar.

3.4. Evaluación/Experimentos

3.4.1. Rendimiento baseline: Agente aleatorio

Primero se definió una línea base para medir el rendimiento mínimo antes de entrenar los algoritmos principales. Para ello se usó un agente aleatorio, que elige acciones de forma uniforme sin aprender ni actualizar parámetros. Como era esperable, su desempeño se mantiene prácticamente constante a lo largo de los episodios, sin tendencia de mejora. En nuestras pruebas, el agente suele no superar la primera tubería (alrededor de 50 pasos) y obtiene recompensas bajas, típicamente entre (-8) y (-7). Este baseline confirma que el entorno no se resuelve por azar y sirve como referencia para cuantificar las mejoras de SARSA semi-gradiente y Double DQN.

`images/aproximadas/BaselineRandom.png`

Figura 3.1: Resultados del modelo baseline en Recompensa y pasos promedio

3.4.2. Influencia de Tile Coding en SARSA

Para el experimento de *Tile Coding* (con SARSA semi-gradiente), fijamos primero una configuración constante y solo variamos la granularidad de discretización para medir su impacto. Como parámetros fijos se mantuvieron: (α=0.01), (γ=0.99), política (ϵ)-greedy ((ϵ_{start}=1.0), ($\epsilon_{\text{mín}}$=0.05), (ϵ_{decay}=0.995)), (n_steps=1), (n_tilings=8), (iht_size=4096), semilla, y el mismo número de episodios de entrenamiento (1500). El único hiperparámetro que se barrió fue (bins $\in$ 6,8,10,12), evaluando cada configuración por el rendimiento medio en los últimos 50 episodios ((*reward_last*50), y como referencia también (steps_last50)). Los resultados muestran diferencias claras

Tabla 3.1: Resultados de los últimos 50 episodios de las distintas configuraciones de Tile Coding

Bins	recompensa	pasos
6	2.6	64.3
8	-9.3	50.0
10	0.16	51.6
12	-8.42	50.0

entre resoluciones: con menos *bins* (estado más agregado) el aprendizaje fue más rápido convergiendo antes hacia una solución con mayores recompensas promedio. La mejor configuración resultó ser (bins=6) (con ($\alpha = 0{,}01$)), que fue la usada posteriormente en la comparación principal frente a DQN. En conjunto, el barrido confirma que en este problema una discretización demasiado fina retrasa la convergencia como se ve en 3.2, y puede introducir mayor varianza sin ganancias de recompensa.

images/aproximadas/SARSATile.png

Figura 3.2: Resultados de SARSA con Tile coding en Recompensa, pasos promedio y puntuación

3.4.3. Influencia de parámetros Deep Q-Learning

En esta fase se analizó el impacto de learning_rate y buffer_capacity en Deep Q-learning, manteniendo constante el resto de parámetros del agente y el entorno, con 1500 episodios por configuración. El objetivo fue aislar el efecto de la dinámica de aprendizaje (tamaño de paso de gradiente) y de la memoria de experiencias sobre el rendimiento final. Como métricas principales se emplearon la recompensa promedio de los últimos 50 episodios y la duración promedio de los últimos 50 episodios, para medir tanto recompensa acumulada como estabilidad en supervivencia. Los resultados muestran

lr	buffer	recompensa	pasos
5e-05	10000	4.530	55.3
5e-05	30000	4.102	51.7
5e-05	50000	4.016	50.8
1e-04	10000	5.352	60.7
1e-04	30000	4.106	51.5
1e-04	50000	4.186	52.3
5e-04	10000	5.490	62.5
5e-04	30000	5.986	66.0
5e-04	50000	4.218	53.1
1e-03	10000	6.332	69.1
1e-03	**30000**	**6.740**	**72.1**
1e-03	50000	4.258	53.2

Tabla 3.2: Resultados de DQN para distintas tasas de aprendizaje y tamaños de buffer.

una tendencia clara: tasas de aprendizaje más altas dentro del rango evaluado (1e-3) obtienen mejor rendimiento final que valores más conservadores (5e-4, 1e-4, 5e-5). En paralelo, una capacidad intermedia del replay buffer (30000) supera tanto a 10000 como a 50000, lo que sugiere un equilibrio entre diversidad de experiencia y capacidad de adaptación a la política actual. En términos prácticos, el mejor punto del barrido fue learning_rate=1e-3 y buffer_capacity=30000, con reward_last50=6.740 y steps_last50=72.10.

3.4.4. Influencia de la arquitectura de la red en Deep Q-Learning

Una vez fijados los mejores hiperparámetros de optimización, se estudió por separado la arquitectura del aproximador profundo variando numero de nodos y capas. Esta segunda fase buscó medir cómo la capacidad representacional de la red afecta a la calidad de la política aprendida, manteniendo el mismo entrenamiento (1500 episodios) y las mismas condiciones experimentales. Se comparó tanto rendimiento temprano como rendimiento final como la media de los últimos 50 episodios

images/aproximadas/ArquitecturaDQN.png

Figura 3.3: Resultados de DQN con distintas arquitecturas

El análisis indica que aumentar capacidad mejora el rendimiento hasta un punto: las redes de una sola capa (32x1, 64x1) quedan claramente por debajo, mientras que arquitecturas de tres capas (64x3, 128x3) capturan mejor la dinámica del en-

torno. Sin embargo, incrementar profundidad adicionalmente (128x4) no mejora resultados e incluso los degrada. La mejor arquitectura fue 128 nodos y 3 capas, alcanzando recompensas promedio en los últimos 50 episodios entre 6.870 y pasos 75.44, por encima del resto de configuraciones evaluadas.

3.4.5. Agregación de estados

El experimento de frame stacking se diseñó para medir cuánto contexto temporal adicional ayuda al agente. En lugar de usar solo la observación actual, se concatenaron (k) observaciones consecutivas (k en 1,3,6) y se repitió el entrenamiento para DQN y SARSA semi-gradiente con el resto de hiperparámetros fijos. Así se aisló el efecto de la memoria temporal sobre la política aprendida.

La hipótesis era que más frames podrían mejorar la toma de decisiones al incorporar tendencia de movimiento (por ejemplo, caída/subida del pájaro y evolución relativa respecto a tuberías). Sin embargo, también aumenta la dimensión del estado y puede complicar la optimización, especialmente en métodos con aproximación lineal y codificación dispersa como SARSA+Tile Coding.

images/aproximadas/AgregacionEstados.png

Figura 3.4: Resultados de DQN y SARSA con distintas agregaciones de los estados

En los resultados, DQN mantuvo buen rendimiento con 1 y 6 frames, pero empeoró de forma clara con 3 frames. Para SARSA semi-gradiente, el efecto fue más sensible con un resultado pésimo con más de 1 frame, esto tambien afecta notablemente el tiempo de ejecución al incrementar el estado.

La lectura que podemos hacer de este experimento es que el frame stacking no garantiza una mejora en este entorno depende del algoritmo y de la forma de representación del estado.

3.4.6. n-Pasos

En la comparativa por n-pasos, ambos algoritmos muestran el mismo patrón general: la mejor configuración es (n=1), y aumentar el horizonte a (n=5) o (n=10) degrada el rendimiento. En DQN, pasar de (n=1) a (n=5) reduce la recompensa de (8.26) a (4.98) y los pasos de (83.98) a (58.70); con (n=10) cae aún más en recompensa ((4.56)) y mantiene episodios cortos ((55.04) pasos). Esto sugiere que, en este entorno, el retorno largo introduce objetivos más ruidosos o menos útiles para la actualización.

SARSA semi-gradiente sigue la misma tendencia, aunque con

images/aproximadas/Npasos.png

Figura 3.5: Resultados de DQN y SARSA con distintos n pasos

una caída más acusada en score: (n=1) alcanza (6.82) de recompensa y (79.06) pasos, mientras que (n=5) baja a (3.95) y (50.47), y (n=10) se queda en (4.07) y (51.31). Además, el score de SARSA prácticamente desaparece para (n>1) ((0.00) y (0.05)), lo que indica que el agente deja de superar tuberías de forma consistente.

Comparando directamente ambos métodos, DQN domina en las tres configuraciones de n-step: obtiene mayor recompensa y mayor longitud de episodio en todos los casos. La diferencia es especialmente clara en (n=1), donde DQN llega a (83.98) pasos frente a (79.06) de SARSA y supera también en recompensa final ((8.26) vs (6.82)). En (n=5) y (n=10), ambos se deterioran, pero DQN conserva una ventaja sistemática.

3.5. Conclusiones y Vías futuras

3.5.1. Conclusiones

Tras el desarrollo y la evaluación de los algoritmos propuestos para el entorno de Flappy Bird, podemos concluir que el aprendizaje por refuerzo profundo, específicamente mediante **Deep Q-Learning (DQN)**, ha demostrado una superioridad clara frente a la aproximación lineal de **SARSA semi-gradiente**.

En términos de rendimiento, DQN no solo ha logrado alcanzar puntuaciones más altas en un menor número de episodios, sino que ha mostrado una **convergencia más estable hacia recompensas positivas**. Mientras que SARSA, apoyado en *Tile Coding*, consiguió superar la línea base aleatoria con éxito, se mostró mucho más sensible a la dimensionalidad del estado y a la configuración de los *bins*. En contraste, la capacidad de generalización de las redes neuronales en DQN permitió al agente extraer mejores características de las 12 variables continuas del entorno.

Respecto a las técnicas adicionales:

- El uso de **n-pasos** y **agregación de estados (frame stacking)** no resultó en una mejora inmediata en este entorno particular. Se observó que aumentar el horizonte de predicción ($n > 1$) o la memoria temporal ($k > 1$) introducía ruido y complejidad que dificultaba el aprendizaje, siendo las configuraciones más simples las más efectivas.
- DQN demostró ser más resiliente ante cambios en la ar-

quitectura, encontrando un "punto dulce.[en] redes de 3 capas y 128 nodos, lo que confirma que el problema requiere una capacidad representacional no lineal moderada.

3.5.2. Limitaciones y vías futuras

A pesar de los resultados satisfactorios, el presente trabajo ha enfrentado limitaciones importantes. La principal limitación técnica fue el coste computacional. Inicialmente, se planteó el uso de un entorno basado en sensores **LiDAR** para la percepción del agente; sin embargo, esta configuración resultó ser excesivamente pesada para el entrenamiento, ralentizando los ciclos de iteración y obligando a la transición hacia el espacio de estados numérico actual, más ligero y rápido.

Como líneas de trabajo futuro, se proponen las siguientes áreas de mejora:

- **Ajuste fino de hiperparámetros entrelazados:** Una vía prometedora sería realizar un estudio de búsqueda de hiperparámetros (Grid o Bayesian Search) que no analice los parámetros de forma aislada, sino su interacción. Por ejemplo, investigar cómo el learning rate debe adaptarse dinámicamente según la profundidad de la arquitectura de la red para evitar la degradación del aprendizaje.
- **Visualización de políticas en alta dimensionalidad:** Debido a que el espacio de estados cuenta con 12 dimensiones (que aumentan linealmente con el *frame stacking*), la interpretación directa de la política óptima es compleja. Se propone el uso de técnicas de reducción de dimensionalidad como t-SNE o UMAP para proyectar las decisiones del agente en un plano 2D, o bien identificar las variables más influyentes mediante análisis de importancia para generar mapas de calor que expliquen el comportamiento del pájaro.
- **Análisis de riesgo:** Sería de interesante cuantificar el 'estilo de vuelo' de cada algoritmo. Esto implicaría medir métricas de **proximidad a los obstáculos**, analizando si el agente prefiere pasar por el centro del hueco o si asume trayectorias más arriesgadas. Esta comparativa permitiría verificar si SARSA, por su naturaleza *on-policy*, desarrolla una política más conservadora y segura en comparación con la agresividad potencial de un agente *off-policy* como DQN. Como pasa en el problema de 'Cliff Walking'.

Bibliografía

[1] Max Chiswick. Monte Carlo RL and Blackjack — chisness.github.io. https://chisness.github.io/2020-09-21/monte-carlo-rl-and-blackjack#importance-sampling-prediction. [Accessed 15-03-2026].

[2] Iveta Dirgová Luptáková, Martin Kubovčík, and Jiří Pospíchal. Playing flappy bird based on motion recognition using a transformer model and lidar sensor. *Sensors*, 24(6), 2024.

[3] M. Kubovčík. Flappy bird gymnasium environment. https://github.com/markub3327/flappy-bird-gymnasium, 2023.

[4] Neel Modha. Reinforcement Learning: Optimal Blackjack Strategies — medium.com. https://medium.com/mitb-for-all/reinforcement-learning-optimal-blackjack-strategies-982a5a176d2d. [Accessed 15-03-2026].

[5] Gabriel Guedes Nogueira. flappy-bird-gym: An openai gym environment for the flappy bird game. https://github.com/Talendar/flappy-bird-gym, 2021.

[6] E. Racah and K. Pal. Self-supervised representations in interactive environments. *arXiv preprint arXiv:1906.11951*, 2019.

[7] Avi Singh, Larry Yang, Kristian Hartikainen, Chelsea Finn, and Sergey Levine. End-to-end robotic reinforcement learning without reward engineering. *arXiv preprint arXiv:1904.07854*, 2019.

[8] Richard S Sutton and Andrew G Barto. *Reinforcement Learning*. Adaptive Computation and Machine Learning series. Bradford Books, Cambridge, MA, 2 edition, November 2018.

[9] T. Vu and L. Tran. Flapai bird: Training an agent to play flappy bird using reinforcement learning techniques. *arXiv preprint arXiv:2003.09579*, 2020.